Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 6

Subject	DBMS Lab
Assignment No.	6
Topic	Database Triggers
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	31/03/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To study and implement different types of database triggers using SQL for a suitable application.

Objectives:

- To understand the concept of triggers in DBMS.
- To implement BEFORE and AFTER triggers for INSERT, UPDATE, and DELETE operations.
- To automate database operations.
- To maintain data integrity and logging.

Case Study: Airline Management System

SQL Queries (Database Triggers):

---

1. BEFORE INSERT Trigger

Theory:
A BEFORE INSERT trigger is executed before inserting a new record. It is used to validate or modify data before it is stored in the table.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_insert_passenger
    -> BEFORE INSERT ON passenger
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.age IS NULL THEN
    ->         SET NEW.age = 18;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

2. AFTER INSERT Trigger

Theory:
An AFTER INSERT trigger is executed after a record is inserted. It is mainly used for logging or audit purposes.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_insert_passenger
    -> AFTER INSERT ON passenger
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO passenger_log VALUES
    ->     (NEW.passenger_id, NEW.pname, 'INSERT');
```

```
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

3. BEFORE UPDATE Trigger

Theory:
A BEFORE UPDATE trigger is executed before updating a record. It helps in validating new values before they are saved.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_update_flight
    -> BEFORE UPDATE ON flight
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.price < 0 THEN
    ->         SET NEW.price = OLD.price;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

4. AFTER UPDATE Trigger
Theory:

An AFTER UPDATE trigger is executed after updating a record. It is useful for maintaining logs or tracking changes.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_update_flight
    -> AFTER UPDATE ON flight
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO flight_log VALUES
    ->     (OLD.flight_id, OLD.price, NEW.price, 'UPDATED');
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

5. BEFORE DELETE Trigger

Theory:

A BEFORE DELETE trigger is executed before deleting a record. It is used to backup data before removal.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_delete_passenger
    -> BEFORE DELETE ON passenger
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO passenger_backup VALUES
    ->     (OLD.passenger_id, OLD.pname, OLD.age);
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

6. AFTER DELETE Trigger

Theory:

An AFTER DELETE trigger is executed after deleting a record. It helps in logging deleted data for audit purposes.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_delete_passenger
    -> AFTER DELETE ON passenger
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO passenger_log VALUES
    ->     (OLD.passenger_id, OLD.pname, 'DELETE');
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Case Study: Civic Voice Interface System

SQL Queries (Database Triggers):

1. BEFORE INSERT Trigger

Theory:
This trigger ensures default values are set before inserting records into the table.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_insert_complaint
    -> BEFORE INSERT ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.status IS NULL THEN
    ->         SET NEW.status = 'Pending';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

2. AFTER INSERT Trigger

Theory:
This trigger logs new complaints after insertion.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_insert_complaint
    -> AFTER INSERT ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO complaint_log VALUES
    ->     (NEW.complaint_id, NEW.issue, 'INSERT');
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

3. BEFORE UPDATE Trigger

Theory:
This trigger ensures that important fields are not updated with invalid values.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_update_complaint
    -> BEFORE UPDATE ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.issue = '' THEN
    ->         SET NEW.issue = OLD.issue;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

4. AFTER UPDATE Trigger

Theory:

This trigger records changes made to complaint status.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_update_complaint
    -> AFTER UPDATE ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO complaint_log VALUES
    ->     (OLD.complaint_id, NEW.status, 'UPDATED');
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

5. BEFORE DELETE Trigger

Theory:

This trigger stores complaint data before deletion to avoid data loss.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER before_delete_complaint
    -> BEFORE DELETE ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO complaint_backup VALUES
    ->     (OLD.complaint_id, OLD.issue, OLD.status);
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

6. AFTER DELETE Trigger

Theory:

This trigger logs deleted complaint records for auditing.

```
mysql> DELIMITER //
mysql> CREATE TRIGGER after_delete_complaint
    -> AFTER DELETE ON complaint
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO complaint_log VALUES
    ->     (OLD.complaint_id, OLD.issue, 'DELETE');
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Conclusion:

In this assignment, different types of triggers such as BEFORE and AFTER triggers for INSERT, UPDATE, and DELETE operations were successfully implemented. These triggers help in automating tasks, ensuring data integrity, and maintaining logs. Hence, triggers are essential components in real-world database applications.